n = 83 K600 rank cut: partner roots by point addition in the rank stage, end-to-end cold

2026-10-16

n = 83 K600 rank cut: partner roots by point addition in the rank stage, end-to-end cold

Status (2026-10-16): protocol frozen; Tier 1 run pending. The protocol and schedule were committed before any Tier 1 timing. The workflow `.github/workflows/n83-rankcut.yml` runs them on one GitHub `ubuntu-24.04` runner. No result is reported here yet. The numbers under "Cost structure", "Local profile" and in `smoke/` come either from the archived baseline run or from the cloud-agent VM. The VM numbers are diagnostics, not timing evidence.

Question

The current n = 83 a = 1 K600 one-target index-calculus baseline is the stacked producer, `koblitz-n83-stacked-20261016` (PR #938, run 38862448935, row in `docs/ic/BASELINES.md`): end-to-end cold cost 283.8 s, `S` 9.68, gap to the tuned rho rT770 94.4 [73.5, 125.3]. Its rank stage takes 257.8 s of the 283.8 s (91%).

The change: the rank stage computes each candidate's two partner roots by point addition instead of the S3 root solve, keeping every relation and every probe. Does that lower the **end-to-end cold** cost on new targets and new rank seeds, with every rank relation, every per-attempt probe count, every scalar and the factor-log table unchanged? This is an iteration under AGENTS.md "Success criterion"; a promoted result becomes the new n = 83 row.

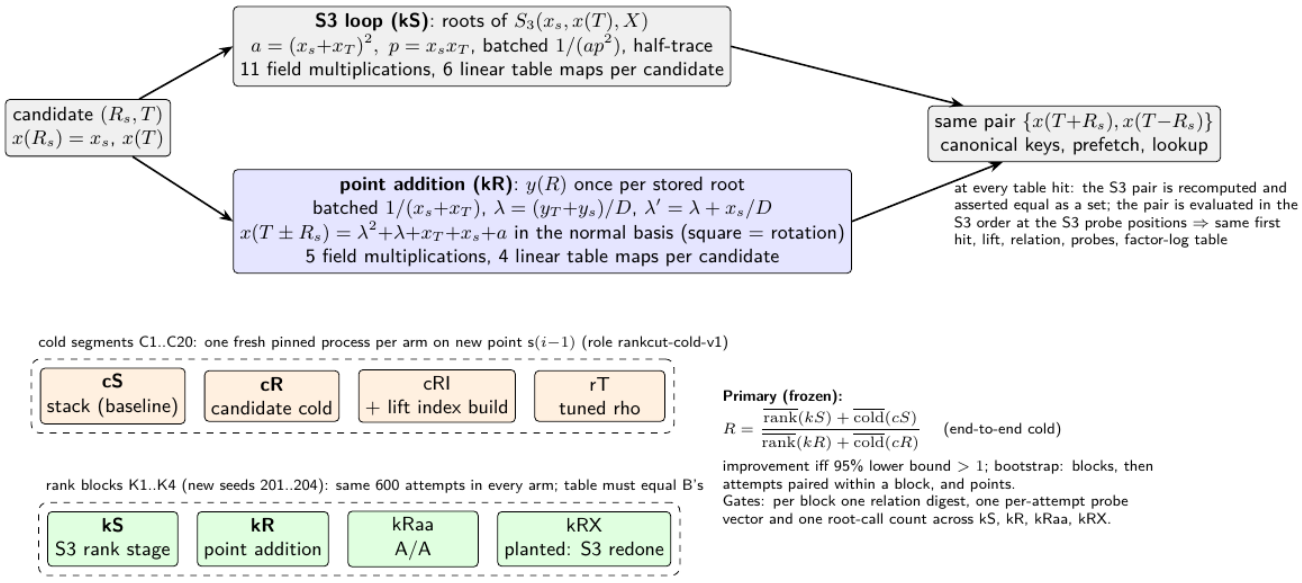


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

Cost structure of the baseline's rank stage (archived run 38862448935)

Source: arm kS, serials 42 and 116 (rank seeds 101 and 102), decoded from the raw records of `koblitz-n83-stacked-20261016/run_github_38862448935`.

Quantity	Value
attempts, relations, rank	600, 600, 600 (per process)
rows without rank gain, failed attempts	0, 0
policy	guided: decompose [a]G - R_j for the first pivotless column j; stop at full rank
relation search share of the stage	255,719 of 255,965 ms (99.9%); matrix build 133 ms, queries 91 ms, relation checks 21 ms, linear algebra 91 ms
probes per attempt	mean 1,667,423 (1.0005e9 per stage), a long-tailed first-hit count (e.g. 248k, 496k, 1.38M, 682k, 1.10M)
ns per probe	rank 255.7; the same process's online target 255.7; cold arm cS online 256.1
root solves per probe	0.5: every candidate has two partners, so the S3 quadratic never fails for a genuine state

So:

- **No attempt is wasted.** The guided minimum-rank policy suggested in the BL-arm row is already the stack's policy, and the stage already stops at full rank. 600 independent rows are the minimum for 600 columns, and the probes per row are fixed by the index (the hit probability per probe), so the number of attempts cannot fall.
- **The rank search is the online loop.** `KIC_RANK_PROBE=target` runs the compact-table loop (`PIPE_COMPACT`): per state, the 2n = 166 candidates are solved in one batch, then all of the state's keys (332 here) are computed, each followed by a prefetch of its home slot, then looked up in order. The pipeline depth is one state, about 332 lookups in flight. Rank and online cost the same per probe (255.7 ns). The stacked run's 32 MiB filter arm (cSF), which skips most table lookups, was 7.2% faster online, so the table lookup is a small part of a probe on the runner. Most of the 256 ns is arithmetic.
- **So the lever is the arithmetic per probe**, mostly the S3 root solve.

Local profile (cloud-agent VM; diagnostic)

From `smoke/profile_local.json` (patch `smoke/profile.patch`). This is the stacked producer on the rank stage's first target, re-running the same 20,000 states (6.64 M probes) with stages added one at a time. The numbers are cumulative ns per probe, three rounds:

Stage	ns per probe
rebuild the folded state	0.3-0.4
+ 2n candidates (`to_poly` of each rotated root)	9-15
+ batched S3 root solve	120-142
+ partners to the normal basis	142-166
+ lift_scan canonical keys	207-213
+ prefetch and compact-table lookup	356-396
the producer's own compact loop on the same states	316-375

The VM has transparent huge pages set to `madvise`, so its 1 GiB table lookups are TLB-bound (about 60 ns each even without the pipeline). The runner uses `always`, and the filter evidence above says lookups are cheap there. On both, the S3 solve is the largest arithmetic block, about 110-130 ns per probe here.

Two cheaper exact replacements were tried in the same harness:

- **Inlining the carry-less multiply** (a `+pclmulqdq` build): about 5-10% on the solve, inside the VM's noise. Not pursued.
- **Partner roots by point addition** (this experiment): the solve drops from 131-163 to 79-95 ns per probe, and the whole loop runs at 231-246 ns per probe against 325-381 for the compact loop in the same runs (about 1.3-1.4x). It matched the S3 pair, as a set, on all 498,000 candidates checked.

The change: `KIC_RANK_SOLVE=padd`

`make_online_source.py` derives the producer in two steps:

1. `make_stacked_source.py` is the stacked experiment's `make_online_source.py`, verbatim, with its three verbatim parents. Its output must hash to the stacked protocol's frozen producer digest (`83b58988...`).
2. Eight counted, exact replacements add `KIC_RANK_SOLVE`, which only the rank stage reads:
   - `s3`, the default, is the stack's rank stage.
   - `padd` replaces the stack's compact loop with `scan_state128_padd`. For each stored root of a state (normal-basis x), the loop recovers y once: `y = x * HT(x + a + b/x^2)`, one inversion and one half-trace. Then, for every Frobenius shift s, R_s = ($x^{(2}$s), $y^{(2}$s)) (two table maps) and one batched inversion of D = x(T) + x(R_s) give `lambda = (y(T) + y(R_s)) / D` and `lambda'=lambda+x(R_s)/D`. The partners are `x(T + R_s)` and `x(T - R_s)` = `lambda^2+lambda+x(T)+x(R_s)+a`, written directly in the normal basis, where squaring is a rotation. That is 5 field multiplications and 4 table maps per candidate, against the S3 path's 11 and 6.
   - `padd_plant` is the planted positive control. It is `padd` plus the removed work: the S3 batch solve and both partners' normal-basis maps on every candidate, compared with the point-addition pair as a set (a panic on any difference), then discarded.

Why it is exact.

- For points R, T of E(F_2^83) with x(R) != x(T), the two roots of S3(x(R), x(T), X) are exactly x(T + R) and x(T - R). Every stored root is the x of a point of E (it is x(P1 +- P2)). The Frobenius conjugates of a point are points of E, because E is defined over F_2. So the pair equals the S3 loop's pair as a set, and the canonical keys are the same values.
- Only the order inside a pair could differ. At every root-table hit the loop:
  - recomputes the candidate's S3 pair with the S3 solver;
  - asserts it equal to the emitted pair as a set;
  - evaluates the pair in the S3 order, at the S3 loop's probe positions.
- So the first hit, the lift, the relation and the probe count of every attempt are the S3 loop's. The rank attempts are a function of the rank seed and the earlier relations, so the echelon and the factor-log table are B's.
- Inputs that are exceptional for the S3 batch (x(R_s) = x(T), x(R_s) = 0 or x(T) = 0) are solved by the S3 solver itself. A state whose root fails the curve check falls back to the compact loop. Neither happened locally.
- The setup, the index and the online target scan are the stacked producer's code, unchanged.

Left out, and why.

- **Multi-column acceptance** (accept a hit that gains rank on any pivotless column) cannot raise hits per probe: every hit already gains rank (0 rows without gain), and the hit rate is fixed by the index.
- **Interleaving K attempts in one pipeline** hides memory latency that the one-state pipeline already hides, and the lookup is a small share on the runner.
- **lift_scan in the index build** is a setup change. It runs here as the cold-only secondary arm `cRI`, because the time budget allows it.

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, and one fresh process per run. Every IC arm runs the 16-byte compact table over the folded index, with lift_scan, the Feistel order and the prefetch pipeline on the online path.

Arm	Kind	Rank solve	Role
`cS`	cold	(none)	**baseline** cold: the stacked producer
`cR`	cold	(none)	**candidate** cold (the same cold code; its own processes)
`cRI`	cold	(none)	optional component: cR with lift_scan in the index build
`rT`	rho	—	tuned rho rT770 (Fold/64/DP6), seed 880000 + point seed
`kS`	rank	`s3`	**baseline** rank stage (the stack's)
`kR`	rank	`padd`	**candidate** rank stage
`kRaa`	rank	`padd`	A/A of kR
`kRX`	rank	`padd_plant`	planted positive control

- **Cold panel.** 20 new public points (`inputs/cold_targets.json`, role `rankcut-cold-v1`, `make_targets.py`). They are checked disjoint from Q0, B's panel and every earlier n = 83 panel of the law, including the stacked run's `stacked-cold-v1`, all regenerated. No scalar is known for any of them.
- **Rank blocks.** `K1-K4`, with new rank seeds 201-204, so 2,400 new rank targets. Each block runs one process of each rank arm: setup, the rank stage to full rank, linear algebra, then one target (s000-s003).
- **Segment order:** O1 (cS and cR on Q0), then C1-C5, K1, C6-C10, K2, C11-C15, K3, C16-C20, K4. The time guard is 300 minutes; the job limit is 350 minutes.

Primary (pre-registered): the end-to-end cold ratio.

- E2E(k, c) = the mean rank stage of rank arm k over the blocks + the mean cold time (setup + one-target online, cached table) of cold arm c over the 20 points.
- R = E2E(kS, cS) / E2E(kR, cR).
- Bootstrap, 10,000 draws: the blocks are resampled; within each drawn block the attempt indices are resampled and used for both kS and kR; the points are resampled, paired. The gates make attempt i the same target with the same probes in both arms. Each process's non-attempt time is kept.
- **Improvement** iff the 95% lower bound of R exceeds 1; regression iff the upper bound is below 1; otherwise not separated. **Invalid** if any gate or control fails or any scheduled row is missing.

Controls.

- A/A: E2E(kR, cR) / E2E(kRaa, cS) and cold cR / cS must both be inside [0.97, 1.03].
- Planted: the per-attempt rank cost kRX / kR must have its lower bound above 1.

Gates.

- Every IC process: the compact layout over the folded index, B's index digests, 29,878,182 entries.
- Per point: one scalar across every IC process and rT (oracle replay), one relation and one probe count across every IC process.
- Q0: the archived relation and scalar with 1,987,852 probes.
- Every rank process: full rank, the arm's `rank_solve` recorded, and a factor-log table equal to B's, log for log.
- **Per block: rank_relations_blake3, the 600-entry per-attempt probe vector and rank_root_calls_total identical across kS, kR, kRaa and kRX.**
- kRX compares every input with the S3 roots.

Secondaries. None of them decides.

- Rank: kS/kR per-attempt cost, overall and by attempt-length tercile; ns per probe per arm; kR/kRaa.
- Cold without rank, setup, online and peak RSS for cS/cR (expected 1).
- The optional component: E2E(kS, cS) / E2E(kR, cRI) and cold cR/cRI.
- The point-addition counters.

- `S` per arm (end-to-end cold, cold without rank, online), delta `S` from cS to cR, the end-to-end cold gap to rT770 and the online gaps.

Power (`power_check.py`, `analysis/power.json`; prior `inputs/power_prior.json`, built by `make_power_prior.py` from run 38862448935's kS/kSaa attempts and cS/cSaa cold times). In the prior, the paired per-attempt A/A noise has a 0.75% sd, and the per-process A/A ratios are 1.0013 and 0.9992. The simulation (200 per cell, 300 bootstrap draws each, the frozen paired block bootstrap) gives power 1.00/1.00/1.00 at an end-to-end effect of 1.03, 1.00/1.00/1.00 at 1.05 (the minimal effect of interest) and 1.00/1.00/1.00 at 1.10 for 2, 3 and 4 blocks, and a rejection rate of 0.040/0.005/0.005 at no effect (1.00). The smallest count passing 0.80 after two Monte Carlo SEs is 2; the design runs 4 blocks (the frozen minimum), which also gives four independent rank seeds against process-level drift. The prior holds only two processes per arm, so its process-level spread is a weak estimate; the A/A control (kR/kRaa, E2E inside [0.97, 1.03]) checks it on the run itself.

Runtime estimate. About 2.3 h including the build (up to about 3 h on a slower runner): rank block about 1,050 s, cold segment about 90 s.

Local validation (cloud-agent VM; diagnostics, no timing claims)

See `smoke/README.md`.

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`make_online_source.py`, `make_stacked_source.py`, `make_tablefilter_source.py`, `make_probepipe_source.py`, `make_combined_source.py`, `online/`, `lift2/`	the rank-cut producer (the stacked derivation verbatim, digest-checked, plus `KIC_RANK_SOLVE`); tuned rho; the two-limb canonicalizer crate
`restore.py`, `check_log_table.py`	restore and digest-check crypto 770f6a69 and every local source; re-verify B's factor-log table
`make_targets.py`, `inputs/cold_targets.json`	the new cold panel
`inputs/archived_identities.json`	archived Q0 identity
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior, simulation, result
`arm.py`, `run_schedule.py`	one arm per pinned process; the schedule runner
`verify_receipt.py`	gates, statistics, decision rule; `result.json`, `result_table.md`
`figures/design.*`	design diagram (TikZ source, PDF, PNG)
`make_figures.py`	result figure (after the run)
`smoke/`	local diagnostics: profile, exactness of full local rank stages, tests, runner smoke
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`